

05 - Asynchroon programmeren

📅 09-09-2026 ⌚ Ongeveer 22 minuten 📄 Ongeveer 3320 woorden

In het vorige hoofdstuk heb je geleerd hoe je een Postgres-database opzet met Docker en hoe je deze kunt beheren. In dit hoofdstuk gaan we die kennis in de praktijk brengen door onze tijdelijke *in-memory* database te vervangen door een volwaardige **Repository** die communiceert met onze nieuwe, echte database.

Daarnaast zul je ontdekken hoe eenvoudig deze overstap is. Dankzij de principes van **abstractie (met interfaces)** en **Dependency Inversion** hoeven we de rest van onze applicatie nauwelijks aan te passen om van database-omgeving te wisselen.

Terugblik & Focus

In het vorige hoofdstuk hebben we onze PostgreSQL-database en pgAdmin via Docker operationeel gemaakt en via EF Core Code-First migraties onze eerste database-tabel klaargezet.

Nu gaan we de vruchten plukken van onze losgekoppelde architectuur. We schrijven een échte LaptopRepository die communiceert met de DbContext. Vervolgens introduceren we asynchroon programmeren (async/await) om ervoor te zorgen dat onze API-threads niet blokkeren tijdens zware database-operaties, waardoor onze applicatie productiever en schaalbaar wordt.

LaptopRepository aanmaken

Maak een nieuwe klasse `LaptopRepository` aan in de map **Repositories** en laat deze de interface `ILaptopRepository` **implementeren**. Laat vervolgens de benodigde *method stubs* automatisch genereren.

Een Repository communiceert rechtstreeks met de databron, zodat de Controller dat niet zelf hoeft te doen. In dit geval is die databron onze `DbContext` : de klasse die fungeert als de tussenpersoon tussen de database en de applicatie.

Onze nieuwe `LaptopRepository` heeft dus een instantie van deze `DbContext` nodig. De `DbContext` is hierdoor een **dependency** (afhankelijkheid) van de repository. Zoals we eerder hebben gezien, maken klassen niet zelf hun eigen Dependencies maar lossen we dit op via **Constructor Injection**:

1. We declareren bovenaan in de klasse een `private _variabele` voor de databasecontext.
2. We vragen deze context op als parameter in de `constructor` .
3. We injecteren de binnengekomen waarde in onze private variabele in de constructor.

```
1  using WebApiDemo.Data;
2  using WebAPIDemo.Models;
3  using WebAPIDemo.Repositories;
4
5  namespace WebApiDemo.Repositories
6  {
7      public class LaptopRepository : ILaptopRepository
8      {
9          private WebAPIDemoContext _context;
10
11          public LaptopRepository(WebAPIDemoContext context)
12          {
13              _context = context;
14          }
15
16          public Laptop Create(Laptop laptop) { throw new
17 NotImplementedException(); }
18
19          public void Delete(int id) { throw new NotImplementedException(); }
20      }
21
22          public List<Laptop> GetAll() { throw new
23 NotImplementedException(); }
24
25          public Laptop? GetById(int id) { throw new
26 NotImplementedException(); }
27
28          public void Update(int id, Laptop laptop) { throw new
29 NotImplementedException(); }
30      }
31 }
```

Een groot voordeel aan Entity Framework is dat we geen complexe queries hoeven te schrijven maar dat we de reeds opgedane Linq kennis kunnen hergebruiken om de CRUD methodes te implementeren.

```
1  using WebApiDemo.Data;
2  using WebAPIDemo.Models;
3  using WebAPIDemo.Repositories;
4
5  namespace WebApiDemo.Repositories
6  {
7      public class LaptopRepository : ILaptopRepository
8      {
9          private readonly \WebAPIDemoContext _context;
10
11          public LaptopRepository(WebAPIDemoContext context)
12          {
13              _context = context;
14          }
15
16          // C - Create: Voeg een nieuwe laptop toe aan de database
17          public Laptop Create(Laptop laptop)
18          {
19              _context.Laptops.Add(laptop);
20              _context.SaveChanges();
21
22              return laptop;
23          }
24
25          // R - Read (All): Haal alle laptops op
26          public List<Laptop> GetAll()
27          {
28              return _context.Laptops
29                  .OrderBy(x => x.Merk)
30                  .ToList();
31          }
32
33          // R - Read (Single): Zoek een specifieke laptop op basis van ID
34          public Laptop? GetById(int id)
35          {
36              return _context.Laptops.Find(id);
37          }
38
39          // U - Update: Pas de gegevens van een bestaande laptop aan
40          public void Update(int id, Laptop updatedLaptop)
41          {
42              Laptop? existingLaptop = _context.Laptops.Find(id);
43          }
```

```
44         if (existingLaptop != null)
45         {
46             // Pas de eigenschappen aan
47             existingLaptop.Merk = updatedLaptop.Merk;
48             existingLaptop.Processor = updatedLaptop.Processor;
49             existingLaptop.RamInGB = updatedLaptop.RamInGB;
50             existingLaptop.Prijs = updatedLaptop.Prijs;
51             existingLaptop.GPU = updatedLaptop.GPU;
52
53             _context.SaveChanges();
54         }
55     }
56
57     // D - Delete: Verwijder een laptop uit de database
58     public void Delete(int id)
59     {
60         Laptop? laptopToDelete = _context.Laptops.Find(id);
61
62         if (laptopToDelete != null)
63         {
64             _context.Laptops.Remove(laptopToDelete);
65             _context.SaveChanges();
66         }
67     }
68 }
69 }
```

Info

👉 Merk op dat de code in deze Repository grote overeenkomsten vertoont met de in-memory repository. Zowel op Collecties (Arrays, Lists, ...) als op Entity Framework kan Linq toegepast wordt, wat leidt tot een groot overlap van syntax. Laten we de verschillen met de in-memory dieper onder de loep nemen. 🔍

Wat gebeurt hier precies?

- `_context.SaveChanges()` : Entity Framework werkt met een Change Tracker. Wanneer je `.Add()` of `.Remove()` aanroept, of wanneer je de eigenschappen van een opgehaald object wijzigt, onthoudt EF dit in het geheugen. **Pas op het moment dat je `_context.SaveChanges()` aanroept**, worden deze wijzigingen vertaald naar SQL-state-ments (INSERT, UPDATE, DELETE) en **naar de database gestuurd**.

- `_context.Laptops.Find(id)` : Dit is een efficiënte EF-methode die specifiek zoekt op de Primary Key (het ID) van de tabel. Als de laptop niet wordt gevonden, geeft deze methode netjes null terug.
- `private readonly` : Het is een **best practice** om de `_context` variabele als readonly te markeren. Dit zorgt ervoor dat de databasecontext na de initialisatie in de constructor per ongeluk nergens anders in de klasse overschreven kan worden.

Tips



Jos de Magazijnier

Je kunt een Repository vergelijken met **Jos de magazijnier**. Jos is een topkracht in het magazijn. Als je tegen hem zegt: "*Jos, haal voor mij de top 10 best verkochte laptops*", dan doet hij dat direct. Zeg je: "*Jos, pas de voorraad van dit model aan*", dan gebeurt dat vlekkeloos.

Jos is echter geen manager; hij neemt geen strategische beslissingen. Net zoals Jos zijn Repositories puur gespecialiseerd in het **ophalen en wegschrijven van data**. Ze voeren zelf geen complexe businesslogica, berekeningen of validaties uit—dat is de taak van de Controllers of de Service-laag.

Veelgebruikte LINQ & Entity Framework Statements

Wanneer je met Entity Framework werkt, gebruik je LINQ (Language Integrated Query) om gegevens op te vragen, te filteren en te manipuleren. Hieronder vind je een overzicht van de meest gebruikte statements met hun SQL-equivalent en een korte beschrijving.

LINQ Methode	Doel	Voorbeeld (C#)
<code>ToList()</code>	Haalt alle records op uit de tabel en zet ze om in een lijst.	<code>_context.Laptops.ToList();</code>
<code>Find(id)</code>	Zoekt zeer efficiënt naar één record op basis van de <i>Primary Key</i> .	<code>_context.Laptops.Find(id);</code>

LINQ Methode	Doel	Voorbeeld (C#)
FirstOrDefault(x => ...)	Geeft het eerste element dat aan de voorwaarde voldoet, of null als er niets wordt gevonden.	<pre>_context.Laptops.FirstOrDefault(l => l.Brand == "Apple");</pre>
Where(x => ...)	Filtret de resultaten op basis van een specifieke voorwaarde (booleaanse expressie).	<pre>_context.Laptops.Where(l => l.Price > 1000).ToList();</pre>
OrderBy(x => ...)	Sorteert de resultaten in oplopende volgorde (A-Z, laag-hoog).	<pre>_context.Laptops.OrderBy(l => l.Price).ToList();</pre>
OrderByDescending(...)	Sorteert de resultaten in aflopende volgorde (Z-A, hoog-laag).	<pre>_context.Laptops.OrderByDescending(l => l.Price).ToList();</pre>
Select(x => ...)	Projecteert of transformeert de data naar een specifiek formaat of specifieke kolommen.	<pre>_context.Laptops.Select(l => l.Brand).Distinct().ToList();</pre>

LINQ Methode	Doel	Voorbeeld (C#)
Any(x => ...)	Controleert of er <i>minstens één</i> record voldoet aan de voorwaarde (geeft true of false).	<pre>_context.Laptops.Any(l => l.Price < 500);</pre>
Count(x => ...)	Telt het aantal records dat aan de (optionele) voorwaarde voldoet.	<pre>_context.Laptops.Count(l => l.Brand == "Dell");</pre>
Include(x => ...)	(<i>Eager Loading</i>) Haalt gerelateerde data (navigation properties) direct mee op via een JOIN.	<pre>_context.Laptops.Include(l => l.Specifications).ToList();</pre>

Tips

IQueryable vs. IEnumerable: Zolang je methodes zoals `Where` , `OrderBy` of `Select` aan elkaar ketent, voert Entity Framework nog geen query uit. De query is op dat moment een `IQueryable` (een blauwdruk). Pas wanneer je een methode aanroept die data effectief opeist — zoals `ToList()` , `FirstOrDefault()` , `Find()` , `Count()` of `Any()` — wordt de LINQ-code vertaald naar SQL en daadwerkelijk afgevuurd op de database. Dit principe noemen we **Deferred Execution** (uitgestelde uitvoering).

Abstractie

Het enige wat we nu nog moeten doen om onze API over te zetten naar een echte database, is de in-memory repository vervangen door onze nieuwe database-variant. Dankzij het gebruik van interfaces hoeven we hiervoor slechts één enkele regel code aan te passen in `Program.cs` : de regel die bepaalt welke implementerende klasse gekoppeld wordt aan de interface `ILaptopRepository`.

De rest van onze applicatie — zoals de `ProductController` — blijft absoluut ongewijzigd. De controller communiceert namelijk uitsluitend met de interface (het contract) `ILaptopRepository`. Het enige wat de controller weet, is *wat* de repository kan doen (bijvoorbeeld `GetAllLaptops()`). De controller heeft geen flauw idee van de manier *waarop* dit achter de schermen gebeurt: of de data nu uit het interne computergeheugen komt of via EF Core uit een PostgreSQL-database wordt gehaald, maakt voor de controller niets uit.

```
1 // OUDE SITUATIE (In-memory):                                csharp
2 //builder.Services.AddScoped<ILaptopRepository, InMemoryRepository>();
3
4 // NIEUWE SITUATIE (Database):
5 builder.Services.AddScoped<ILaptopRepository, LaptopRepository>();
```

Doordat klassen op deze manier uitsluitend het absolute minimum over elkaar weten, kunnen ze optimaal met elkaar samenwerken zonder rigide afhankelijkheden. Dit is de enorme kracht van `loosely coupled code`. Het maakt je applicatie **flexibel, eenvoudig uit te breiden en uiterst onderhoudsvriendelijk**. Mocht je over drie jaar willen overstappen naar een heel ander type database (zoals MongoDB of SQL Server), dan hoef je wederom alleen een nieuwe repository te schrijven en die ene regel in `Program.cs` aan te passen.

Sla je veranderingen op en test je applicatie opnieuw uit. Als alles goed gelopen is zou je applicatie nog steeds moeten werken als voorheen, met als grote verandering dat wijzigingen nu opgeslagen blijven, ookal wordt de applicatie afgesloten en opnieuw opgestart!

Async - Await

Databaseoperaties, zoals het ophalen van grote hoeveelheden gegevens, kunnen soms veel tijd in beslag nemen. In de eerdere voorbeelden wordt de code synchroon uitgevoerd. Dit wil zeggen dat elke regel code pas uitgevoerd wordt als de vorige regel code volledig klaar

is.

In de programmeerwereld wordt vaak de term `thread` gebruikt. Een thread kun je beschouwen als een wachtrij aan de kassa: je moet wachten tot je aan de beurt bent en elke klant wordt één voor één afgewerkt. Sommige applicaties ondersteunen `multi-threading`, wat betekent dat meerdere processen parallel naast elkaar kunnen draaien (net alsof je meerdere kassa's in de winkel opent).



Het knelpunt van synchrone code

Hoewel een applicatie meerdere threads kan hebben, kan een thread met synchrone code absoluut maar 1 request tegelijkertijd verwerken. Zolang die thread wacht op een antwoord van de database, is hij letterlijk "bevroren" en kan hij in de tussentijd niets anders doen. Krijgt je applicatie opeens veel zware aanvragen tegelijk? Dan bevroren al je kassa's één voor één. Zodra alle kassa's bezet zijn, komt je hele applicatie tot stilstand en moet elk nieuw inkomend request noodgedwongen in de wachtrij gaan staan.

In een native applicatie zoals een Console, Maui of WinForms zorgt het onnodig lang vasthouden van een thread ervoor dat het lijkt alsof je applicatie is vastgelopen, aangezien het verversen van de UI en de back-end code standaard op dezelfde thread (de UI thread) draaien. Zie onderstaande video voor een live voorbeeld.

Figuur 1: Uitleg asynchroon programmeren

Het voorbeeld uit het filmpje kun je terug vinden op [deze link](#)

In een Web API applicatie zorgt synchrone code er weliswaar voor dat de server meerdere requests kan verwerken via de Thread Pool, maar zodra de belasting stijgt en alle beschikbare threads bezet zijn door blokkerende databasequery's, kan de server geen nieuwe requests meer gelijktijdig afhandelen. Dit fenomeen noemen we `Thread Starvation` en het is een grove verspilling van de server-resources.

Om te voorkomen dat je met een zware databaseoperatie een thread onnodig lang vasthoudt (en zo je applicatie vertraagt), kun je `async` en `await` toepassen. Om een asynchrone methode uit te voeren moeten we `await` voor het uitvoeren van de methode plaatsen. `Await` kan enkel gebruikt worden in een `async` methode. Deze kernwoorden zijn onafscheidelijk van elkaar.



Het returntype Task<T> en Task

Het returntype van een asynchrone methode is vrijwel altijd een `Task<T>` of een `Task`. Een `Task` kun je zien als een belofte: de methode zegt "ik ben nu gestart, en ik beloof je dat ik later het resultaat (van type T) zal opleveren".

- Had je synchrone methode een return type `int`? Dan wordt de asynchrone variant `Task<int>`.
- Had je synchrone methode een `Laptop` als return type? Dan wordt de asynchrone variant `Task<Laptop>`.
- Had je synchrone methode geen return type (`void`)? Dan wordt de asynchrone variant `Task`.

```
1 public Laptop GetById(int id)                                csharp
2 {
3     // De thread wordt hier volledig geblokkeerd totdat de database klaar
4     is
5     return _context.Laptops.Find(id);
6 }
```

```
1 public async Task<Laptop> GetByIdAsync(int id)              csharp
2 {
3     // De thread wordt hier vrijgegeven. Zodra de database het resultaat
4     geeft, hervat de code vanaf dit punt.
5     return await _context.Laptops.FindAsync(id);
6 }
```

Notitie

Gebruik in asynchrone methodes nooit `async void`, behalve bij Event Handlers. Als een `async void` methode crasht, kan de applicatie de fout niet netjes opvangen, wat vaak leidt tot een onverwachte crash van de gehele applicatie. Gebruik steeds enkel `async Task<T>`

Het Domino-effect: Async is Besmettelijk

Een cruciale regel bij het schrijven van asynchrone code in .NET is: **Asynchrone methodes kunnen alleen opgeroepen worden door andere asynchrone methodes**. Wanneer je ergens diep in je applicatie (bijvoorbeeld in je Repository) een asynchrone database-operatie wilt uitvoeren met `await`, móét de methode waarin dit gebeurt gemarkeerd worden met het `async` keyword. Dit heeft direct gevolgen voor de methode die dáár weer boven staat.

Dit principe noemen we vaak "async all the way" of het domino-effect van async:

- Wil je in de Repository een asynchrone query uitvoeren? Dan moet de Repository-methode `async Task` of `async Task<T>` worden.
- Omdat de Repository-methode nu een Task teruggeeft, moet de Interface (`ILaptopRepository`) aangepast worden om deze Task te reflecteren.
- De Controller (of Service-laag) die de Repository aanroept, moet deze aanroep nu ook voorzien van het `await` keyword.
- Hierdoor moet de Controller-actie zélf ook veranderd worden naar een `async Task<ActionResult<...>>`.

Notitie

Probeer dit domino-effect niet te doorbreken door bijvoorbeeld `.Result` of `.Wait()` te gebruiken op een asynchrone taak om hem geforceerd synchroon te maken. Dit heft alle voordelen van asynchroon programmeren op en leidt in veel gevallen tot een permanente blokkering van je threads (een `deadlock`). Hou het simpelweg bij async en `await` tenzij je écht niet anders kan.

Entity Framework Core: Out-of-the-box Async

Gelukkig hoef je niet zelf het wiel uit te vinden om je data-architectuur asynchroon te maken. Entity Framework Core biedt *out-of-the-box* asynchrone tegenhangers voor vrijwel elke LINQ-methode die data daadwerkelijk opeist van de database.

Zodra je de namespace `using Microsoft.EntityFrameworkCore;` boven je bestand hebt staan, krijg je direct toegang tot deze asynchrone methodes. Het omzetten van een synchrone Repository naar een asynchrone Repository is hierdoor extreem eenvoudig en snel te realiseren.

Hier zie je hoe de meest gebruikte synchrone EF-methodes direct overgezet kunnen worden:

Synchroon	Asynchroon
<code>_context.SaveChanges()</code>	<code>await _context.SaveChangesAsync()</code>
<code>_context.Laptops.ToList()</code>	<code>await _context.Laptops.ToListAsync()</code>
<code>_context.Laptops.FirstOrDefault(...)</code>	<code>await _context.Laptops.FirstOrDefaultAsync(...)</code>
<code>_context.Laptops.Any(...)</code>	<code>await _context.Laptops.AnyAsync(...)</code>
<code>_context.Laptops.Count()</code>	<code>await _context.Laptops.CountAsync()</code>
<code>_context.Laptops.Count(id)</code>	<code>await _context.Laptops.FindAsync(id)</code>

Repository async maken

Dankzij deze kant-en-klare methodes hoeven we enkel de return-types in onze interfaces en klassen in te pakken in een `Task<>` en de juiste EF-functies aan te roepen om de volledige database-laag van onze applicatie schaalbaar en performant te maken. Pas

LaptopRepository.cs aan zoals hieronder:

```
1  using Microsoft.EntityFrameworkCore; // Nodig voor de asynchrone      csharp
2  extensiemethodes
3  using WebApiDemo.Data;
4  using WebAPIDemo.Models;
5  using WebAPIDemo.Repositories;
6
7  namespace WebApiDemo.Repositories
8  {
9      public class LaptopRepository : ILaptopRepository
10     {
11         private readonly WebAPIDemoContext _context;
12
13         public LaptopRepository(WebAPIDemoContext context)
14         {
15             _context = context;
16         }
17
18         // C - Create: Voeg een nieuwe laptop toe
19         public async Task<Laptop> CreateAsync(Laptop laptop)
20         {
21             _context.Laptops.Add(laptop);
22             await _context.SaveChangesAsync();
23
24             return laptop;
25         }
26
27         // R - Read (All): Haal alle laptops op
28         public async Task<List<Laptop>> GetAllAsync()
29         {
30             return await _context.Laptops
31                 .OrderBy(x => x.Merk)
32                 .ToListAsync();
33         }
34
35         // R - Read (Single): Zoek een specifieke laptop op basis van ID
36         public async Task<Laptop?> GetByIdAsync(int id)
37         {
38             return await _context.Laptops.FindAsync(id);
39         }
40
41         // U - Update: Pas de gegevens aan
```

```
42     public async Task UpdateAsync(int id, Laptop updatedLaptop)
43     {
44         Laptop? existingLaptop = await _context.Laptops.FindAsync(id);
45
46         if (existingLaptop != null)
47         {
48             existingLaptop.Merk = updatedLaptop.Merk;
49             existingLaptop.Processor = updatedLaptop.Processor;
50             existingLaptop.RamInGB = updatedLaptop.RamInGB;
51             existingLaptop.Prijs = updatedLaptop.Prijs;
52             existingLaptop.GPU = updatedLaptop.GPU;
53
54             await _context.SaveChangesAsync();
55         }
56     }
57
58     // D - Delete: Verwijder een laptop
59     public async Task DeleteAsync(int id)
60     {
61         Laptop? laptopToDelete = await _context.Laptops.FindAsync(id);
62
63         if (laptopToDelete != null)
64         {
65             _context.Laptops.Remove(laptopToDelete);
66             await _context.SaveChangesAsync();
67         }
68     }
69 }
```

::: hint

Schrijf enkel await voor de uit te voeren asynchrone methode en laat Visual Studio (Code) het async keyword en het return type automatisch genereren. Dat bespaart tijd en fouten!

:::

Vergeet ook niet om de **ILaptopRepository.cs** Interface aan te passen. Het contract moet steeds kloppen langs beide kanten!

```
1  using WebAPIDemo.Models;
2
3  namespace WebApiDemo.Repositories
4  {
5      public interface ILaptopRepository
6      {
7          Task<Laptop> CreateAsync(Laptop laptop);
8
9          Task DeleteAsync(int id);
10
11         Task<List<Laptop>> GetAllAsync();
12
13         Task<Laptop?> GetByIdAsync(int id);
14
15         Task UpdateAsync(int id, Laptop updatedLaptop);
16     }
17 }
```

::: hint

Het is een best practice om asynchrone methode de -async suffix te geven in de naam. Zo is het voor andere developers (en je toekomstige zelf) eenvoudig om te zien welke methodes asynchroon zijn en welke niet.

:::

Controller async maken

Nu onze Repository en Interface volledig asynchroon zijn, is het tijd om de laatste schakel aan te passen: de Controller .

Omdat onze controller nu ILaptopRepository aanroept (die methodes heeft met de suffix Async en die een Task retourneren), moeten we in de controller ook gebruikmaken van await.

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebApiDemo.Repositories;
3  using WebAPIDemo.Models;
4
5  namespace WebAPIDemo.Controllers;
6
7  [ApiController]
```

```
8      [Route("api/[controller]")]
9      public class ProductController : ControllerBase
10     {
11         // Onze "ingrediënten" (Dependencies)
12         private readonly ILaptopRepository _laptopRepository;
13         private readonly ILogger<ProductController> _logger;
14
15         // Vraag om en injecteer Dependencies
16         public ProductController(ILaptopRepository laptopRepository,
17     ILogger<ProductController> logger)
18         {
19             _laptopRepository = laptopRepository;
20             _logger = logger;
21         }
22
23         [HttpGet]
24         public async Task<ActionResult<List<Laptop>>> GetAlleLaptopsAsync()
25         {
26             _logger.LogInformation("GET request ontvangen voor alle
27     laptops.");
28             return Ok(await _laptopRepository.GetAllAsync());
29         }
30
31         [HttpGet("{id}")]
32         public async Task<ActionResult<Laptop>> GetLaptopByIdAsync(int id)
33         {
34             _logger.LogInformation($"GET request voor laptop met ID: {id}");
35
36             Laptop? laptop = await _laptopRepository.GetByIdAsync(id);
37
38             if (laptop == null)
39             {
40                 _logger.LogWarning($"Laptop met ID {id} werd niet gevonden.");
41                 return NotFound($"Helaas, we konden geen laptop vinden met Id
42     {id}.");
43             }
44
45             return Ok(laptop);
46         }
47
48         [HttpPost]
49         public async Task<ActionResult<Laptop>> CreateLaptopAsync(Laptop
50     nieuweLaptop)
```

```
51     {
52         _logger.LogInformation($"Aanmaken van nieuwe laptop:
53 {nieuweLaptop.Merk}");
54
55         Laptop aangemaakteLaptop = await
56 _laptopRepository.CreateAsync(nieuweLaptop);
57
58         _logger.LogInformation($"Laptop succesvol aangemaakt met ID:
59 {aangemaakteLaptop.Id}");
60         return CreatedAtAction(nameof(GetLaptopByIdAsync), new { id =
61 aangemaakteLaptop.Id }, aangemaakteLaptop);
62     }
63
64     [HttpPut("{id}")]
65     public async Task<ActionResult> UpdateLaptopAsync(int id, Laptop
66 bijgewerkteLaptop)
67     {
68         _logger.LogInformation($"Update request voor laptop met ID:
69 {id}");
70
71         Laptop? bestaandeLaptop = await
72 _laptopRepository.GetByIdAsync(id);
73
74         if (bestaandeLaptop == null)
75         {
76             _logger.LogWarning($"Update mislukt: Laptop met ID {id}
77 bestaat niet.");
78             return NotFound($"Kan geen laptop bijwerken met Id {id}, omdat
79 deze niet bestaat.");
80         }
81
82         await _laptopRepository.UpdateAsync(id, bijgewerkteLaptop);
83
84         _logger.LogInformation($"Laptop met ID {id} is succesvol
85 bijgewerkt.");
86         return NoContent();
87     }
88
89     [HttpDelete("{id}")]
90     public async Task<ActionResult> DeleteLaptopAsync(int id)
91     {
92         _logger.LogInformation($"Delete request voor laptop met ID:
93 {id}");
```

```
Laptop? laptop = await _laptopRepository.GetByIdAsync(id);

if (laptop == null)
{
    _logger.LogError($"Delete mislukt: Laptop met ID {id} werd niet gevonden.");
    return NotFound($"Kan laptop met Id {id} niet verwijderen, omdat deze niet is gevonden.");
}

await _laptopRepository.DeleteAsync(id);

_logger.LogInformation($"Laptop met ID {id} is succesvol verwijderd.");
return NoContent();
}
```

Notitie

✂ Onze In-memory repository volgt niet langer het contract vooropgesteld door `ILaptopRepository`. Aangezien we geen nut meer hebben voor deze klasse mag deze verwijderd worden.

Samenvatting

- **Dependency Injection (DI)** : Injecteer je `DbContext` in de Constructor.
- **Repository Pattern** : De Repository is de enige plek waar database-logica woont. Houd je Controllers "dun" en laat ze enkel communiceren via de Repository interface.
- ** Async/Await **
 - Gebruik `async Task<T>` voor methodes met returnwaarde, `async Task` voor void.
 - Gebruik `await` bij elke async operatie (database, API-calls).
 - `Await` kan enkel gebruikt worden in `async methodes`
 - Volg de "Async all the way " regel: van de database tot in de Controller.

- **Entity Framework & LINQ** : Je weet nu dat `.ToList()`, `.FirstOrDefault()` en `SaveChanges()` de acties zijn die de database écht aan het werk zetten (`Deferred Execution`).
- **Naming Conventions** : Gebruik consequent de `-Async` suffix voor je methodes. Dit is de standaard in de .NET-industrie.
- **Performance** : Je begrijpt nu het verschil tussen synchroon (wachten en verspillen van threads) en asynchroon (slim vrijgeven van threads), wat cruciaal is voor de schaalbaarheid van je API.

Kortom : Zorg dat je code vanaf nu altijd losgekoppeld, asynchroon en consequent benoemd is. Dit vormt het fundament voor de complexere database-relaties die we in de komende hoofdstukken gaan behandelen.

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM